

USRP SDR Platform — Beginner's Guide

Contents

USRP SDR Platform — Beginner's Guide	1
1. Quick start	1
2. Where things live	1
3. Running an algorithm over the PHY	2
4. Add your own algorithm	2
5. Driving the radio directly (sdr.py / radio.sh)	4
6. The building blocks (pyphy)	6
7. Function cheat-sheet (the Python API)	6
8. Troubleshooting (beginner gotchas)	6
Beyond this guide	7

USRP SDR Platform — Beginner's Guide

A single, self-contained guide: what the platform is, how to run it, how to add your own algorithm, and how to drive the radio directly. No prior knowledge of the codebase needed.

PHY = the "modem". It turns your data (bytes / a float32 vector) into radio waves and back: framing -> error-correction (FEC) -> modulation -> radio, and the reverse on receive. You bring an *algorithm*; the PHY moves its data over the air.

1. Quick start

```
./run.sh                # run an algorithm over the PHY (defaults: echo, no radio)
./run.sh list           # list the algorithms available
./run.sh --algo marl    # run a specific algorithm
./run.sh --help         # help + every option
./radio.sh rx           # raw receive on a USRP (B210 default; --device n210|x310)
./radio.sh tx           # raw transmit
```

Anything you don't specify takes a **default** (algo=echo role=loopback channel=ideal steps=5). run.sh prints the exact command it runs (the >> line) so you learn the underlying call.

2. Where things live

- **algorithms/** — your code (one folder per algorithm); run.sh runs it.
- **phy/build/sdr_system** — the C++ radio engine (the modem); radio.sh runs raw TX/RX.
- **phy/python/** — the Python you call: run_algo.py/phy_link.py (uniform API), sdr.py/run.py/configs/ (direct radio), channel_sense.py/freq_scan.py (RF utils).
- **phy/bindings/pyphy** — the DSP blocks as numpy functions.
- **applications/** — worked apps (MARL, FL, CLIP semantic comm, jammer).

Three ways to use the PHY, easiest first:

1. **Run an algorithm** with `run.sh` (§3).
2. **Add your own algorithm** (§4).
3. **Drive the radio directly** with `sdr.py` / `radio.sh` (§5), or compose the DSP blocks (§6).

3. Running an algorithm over the PHY

Your algorithm only says **what to transmit** and **what to receive** — no radio code. Drop it in `algorithms/<name>/app.py` (copy `algorithms/_template/`), then run it by name:

```
# radio-free, lossless (check the logic):
./run.sh --algo <name>

# radio-free THROUGH THE REAL MODEM + noise (see how SNR affects it):
./run.sh --algo <name> --channel pyphy --snr-db 6

# over the radio (two hosts) - start the rx FIRST:
./run.sh --algo <name> --role rx --rx-args addr=192.168.20.2
./run.sh --algo <name> --role tx --tx-args serial=30CD424 --ack-host <AP_IP>
```

run.sh options

Option	Default	What it does
<code>--algo <name></code>	echo	which folder under <code>algorithms/</code> to run
<code>--role loopback tx rx</code>	loopback	loopback = both ends in one process (no radio); tx/rx = the two radio hosts
<code>--channel ideal pyphy</code>	ideal	loopback channel: ideal = lossless; pyphy = real modem + AWGN
<code>--steps <n></code>	5	how many round-trips to run
<code>--scheme <NAME></code>	QPSK	modulation for the pyphy/radio path (QPSK, BPSK, DQPSK, 16-QAM, ...)
<code>--fec conv ldpc turbo ""</code>	turbo	error-correcting code for the pyphy channel
<code>--snr-db <dB></code>	8	signal-to-noise ratio for the pyphy channel
<code>--tx-args / --rx-args</code>	""	which radio: serial=30CD424 (B210) or addr=192.168.20.2 (N210)
<code>--ack-host / --net-host</code>	127.0.0.1	the peer host's IP (radio roles)

To write your own `app.py`, see §4.

4. Add your own algorithm

Four steps. You write **one small file** (`app.py`); the framework handles the radio.

Step 1 — Where to put it

Create a folder under `algorithms/` whose **name is what you'll pass to `run.sh`**, with an `app.py`:

```
Hardware_update/
└─ algorithms/
   └─ my_algo/          <- the folder name = the algorithm name
      └─ app.py         <- REQUIRED. the framework looks for exactly this file
```

Then run it: `./run.sh --algo my_algo`. (`--algo <name>` must match the folder name; `./run.sh list` shows all algorithms found.)

Step 2 — What `app.py` must provide

`app.py` must define `make(role)` returning an object with two methods:

```
def make(role):          # role is "tx" or "rx" (the framework sets it)
    return YourObject(role)
```

Method / field	Required?	What it is
<code>transmit()</code>	yes	returns the numpy array to send. Return <code>None</code> when there's nothing left (ends the run).
<code>receive(msg)</code>	yes	called with the received numpy array — feed it into your algorithm.
<code>spec = (dtype, shape)</code>	optional	declares your output, e.g. <code>("float32", (16,))</code> .
<code>on_result(ack)</code>	optional	called with <code>True/False</code> = delivered/lost (a reinforcement-learning reward).

No radio code, no imports from the framework.

Step 3 — Two ways to write `app.py`

3A. Simplest — write the algorithm inline (copy `algorithms/_template/`):

```
# algorithms/my_algo/app.py
import numpy as np

class MyAlgo:
    spec = ("float32", (8,))
    def __init__(self, role):
        self.role = role
    def transmit(self):
        return np.ones(8, np.float32)      # what to transmit (None = done)
    def receive(self, msg):
        print("got", msg)                  # what to receive

def make(role):
    return MyAlgo(role)
```

3B. Link your OWN existing algorithm (copy `algorithms/plain_echo/`). Leave your algorithm **untouched** in its own file and let `app.py` just map its methods:

```
algorithms/my_algo/
├─ my_model.py      <- YOUR existing code (unchanged)
└─ app.py           <- the 10-line binding
```

```
# my_model.py — YOUR algorithm. Knows nothing about radios.
import numpy as np
class MyModel:
    def __init__(self):
        self.buf = [np.array([i, i+1, i+2], np.float32) for i in range(5)]
        self.k = 0
    def next_output(self):          # your method that produces data
        if self.k >= len(self.buf): return None
        out = self.buf[self.k]; self.k += 1; return out
    def take_input(self, x):        # your method that consumes data
        print("  my_model received:", x)

# app.py — the binding: map YOUR methods onto transmit()/receive()
import os, sys
sys.path.insert(0, os.path.dirname(os.path.abspath(__file__))) # import files in this folder
from my_model import MyModel
```

```
def make(role):
    model = MyModel()
    class Bind:
        spec = ("float32", (3,))
        def __init__(self): self.role = role
        def transmit(self):
            return model.next_output()      # <- your algorithm's output goes on the air
        def receive(self, msg):
            model.take_input(msg)           # <- the received array goes into your algorithm
    return Bind()
```

Point transmit/receive at your algorithm's real method names — that's the "link". Extra packages (numpy, torch, ...)? Just pip install them; the binding imports your code as-is.

Step 4 — tx vs rx (only if your two ends differ)

Each node is built with a role: **tx** (starts by sending) or **rx** (starts by receiving, may reply). If both ends do the same thing, ignore role; if they differ, branch on it:

```
def transmit(self):
    if self.role == "rx":
        return np.array([self.last_answer], np.float32)  # rx's reply
    return self.next_request()                          # tx's request
def receive(self, msg):
    if self.role == "rx":
        self.last_answer = my_model.process(msg)        # rx handles the request
    else:
        my_model.apply(msg)                             # tx handles the reply
```

(See algorithms/clip_semcom/ — tx sends an image embedding, rx classifies it and replies the label.)

Step 5 — Run it

```
./run.sh --algo my_algo                                # radio-free, lossless
./run.sh --algo my_algo --channel pyphy --snr-db 6      # radio-free, real modem + noise
./run.sh --algo my_algo --role rx --rx-args addr=192.168.20.2 # over the radio (rx first)
./run.sh --algo my_algo --role tx --tx-args serial=30CD424 --ack-host <RX_IP>
```

Checklist / common mistakes

- ☐ Folder is algorithms/<name>/ and --algo <name> matches it exactly.
- ☐ app.py exists and defines make(role).
- ☐ transmit() returns a **numpy array**, or None to stop.
- ☐ Build state inside make() (per node) — **not** module-level globals, or the two loopback ends share state.
- ☐ Importing your own file? add sys.path.insert(0, os.path.dirname(os.path.abspath(__file__))) (as in 3B).
- ☐ spec is optional — the wire format is self-describing, so shapes/dtypes are carried for you.

5. Driving the radio directly (sdr.py / radio.sh)

When you want raw control of the modem (no algorithm layer), use the roles.

Shortcut: ./radio.sh tx or ./radio.sh rx runs a raw TX/RX on a USRP with the right per-device defaults — **B210 is the default**; add --device n210 or --device x310. E.g. ./radio.sh rx --device n210 --freq 915e6. Add --dry-run to just print the command.

From Python (phy/python/sdr.py)

```
from sdr import tx, rx, both, source_arq, sink_arq, run_pair, options
options()                                # print every option + default
tx(tx_args="serial=30CD424", scheme="QPSK", tx_gain=78, fec=True).run() # transmit only
rx(rx_args="serial=30CD3F7", scheme="QPSK", fec=True).run()           # receive only
# reliable stop-and-wait ARQ across two radios (RX first):
run_pair(sink_arq(rx_args="serial=30CD3F7", scheme="QPSK", fec=True),
         source_arq(tx_args="serial=30CD424", scheme="QPSK", fec=True))
print(tx(scheme="QPSK").command())      # just print the command (don't run)
```

The roles

Role	Meaning
tx / rx	transmit only / receive only (one radio each)
both	transmit and receive in one process (single-box loopback)
source_arq / sink_arq	the two ends of reliable stop-and-wait ARQ (with ACK)
sense	listen and report channel power (used by channel_sense.py)

The options you'll actually use (curated — the full list is `PARAMETERS.md`, or `sdr_system --help`)

Option	Example	Meaning
<code>--role</code>	<code>source_arq</code>	what this process does (table above)
<code>--scheme</code>	<code>QPSK</code>	modulation: BPSK QPSK 8-PSK 16/64-QAM, differential DBPSK DQPSK 8-PSK
<code>--waveform</code>	<code>sc</code>	<code>sc</code> (single-carrier) or <code>ofdm</code> (many subcarriers, tracks frequency drift)
<code>--fec</code>	<code>true</code>	turn error-correction on
<code>--fec-type</code>	<code>turbo</code>	<code>conv</code> (fast), <code>ldpc</code> , or <code>turbo</code> (strongest)
<code>--fec_soft</code>	<code>true</code>	soft-decision decoding (better; pair with <code>ldpc/turbo</code>)
<code>--tx-args / --rx-args</code>	<code>serial=30CD424 / addr=192.168.20.2</code>	pick the radio (B210 serial / N210 addr)
<code>--tx-freq / --rx-freq</code>	<code>915e6</code>	carrier frequency (Hz)
<code>--tx-rate / --rx-rate</code>	<code>2e6</code>	sample rate (Hz); N210 needs <code>2e6</code>
<code>--symbol-rate</code>	<code>1e6</code>	symbol rate; N210 pairs <code>2e6/1e6</code>
<code>--tx-gain / --rx-gain</code>	<code>78 / 30</code>	amplifier gain (dB)
<code>--tx-subdev / --rx-subdev</code>	<code>A:A / A:0</code>	radio front-end (B210 A:A, N210 A:0)
<code>--ack-transport</code>	<code>tcp</code>	send the ARQ ACK over <code>tcp</code> (so an RX-only radio never transmits)
<code>--ack-host / --ack-port</code>	<code>192.168.1.50 / 5599</code>	where the ACK goes
<code>--bytes-length</code>	<code>125</code>	payload size per frame (bytes)
<code>--max-attempts</code>	<code>1</code>	retransmit limit; <code>1</code> = single-shot, <code>0/high</code> = keep trying
<code>--viz / --viz-dir</code>	<code>true / phy_outputs</code>	capture TX/RX signals + auto-plot to <code>phy_outputs/<scheme>/</code>
<code>--config <file></code>	<code>phy.cfg</code>	load all options from a file (CLI still overrides)

Two rules that save hours: **start the receiver before the transmitter**, and on free-running radios use **DQPSK** (single-carrier) or **OFDM** — plain coherent QPSK delivers poorly without a shared clock.

From a JSON file (phy/python/run.py)

```
python3 phy/python/run.py --list          # ready-made configs
python3 phy/python/run.py configs/qpsk_arq_pair.json --dry-run # print, don't run
```

6. The building blocks (pyphy)

Every DSP stage is also a plain Python function, so you can compose the chain yourself (GNU-Radio style). Build it with `phy/bindings/build.sh`; import with `PYTHONPATH=phy/bindings python3` (add `arch -x86_64` on macOS).

Group	Functions
Framing	<code>frame, unframe</code>
FEC	<code>fec_encode, fec_decode, fec_decode_soft, fec_encoded_len (conv ldpc turbo, k)</code>
Modulation	<code>modulate, demodulate, soft_llr</code>
Single-carrier	<code>rrc_tx, rrc_rx (pulse shape / matched filter)</code>
Sync	<code>preamble, acq, cfo_correct, phase_correct</code>
OFDM	<code>ofdm_mod, ofdm_demod, ofdm_data_per_sym</code>
Radio (UHD build)	<code>Radio(role, args, freq, rate, symbol_rate, gain, subdev, ant) .transmit() .capture()</code>

```
import numpy as np, pyphy
bits = (np.random.rand(2000) > 0.5).astype(np.uint8)
syms = pyphy.modulate(bits, "QPSK")           # bits -> symbols
syms = syms * 0.6                             # <- your own op between stages
wave = pyphy.rrc_tx(syms, sps=2, beta=0.25)    # pulse-shape to a waveform
# ... channel/radio ...
back = pyphy.demodulate(pyphy.rrc_rx(wave, sps=2, beta=0.25)[:2], "QPSK")
```

Worked flowgraph: `phy/python/phy_flow_example.py`.

7. Function cheat-sheet (the Python API)

Uniform API — `phy/python/phy_link.py`

- `adapt(obj, role)` — wrap ANY object with `transmit()/receive()` into the internal app.
- `PayloadSpec(dtype, shape)` — declare an output's type + shape.
- `Codec.pack(arr) / Codec.unpack(buf)` — numpy array <-> self-describing bytes.
- `IdealChannel / PyphyChannel(scheme, fec, snr_db)` — radio-free channels.
- `run_loopback(tx, rx, channel, steps)` — radio-free round-trip driver.
- `RadioRoundTrip(role, ...).step(app)` — the two-host radio round-trip.

Radio wrapper — `phy/python/sdr.py`

- `SDR(**opts)` — one invocation; `.command()` (print), `.run()` (blocking), `.popen()` (background), `.set(**opts)`.
- helpers `tx / rx / both / source_arq / sink_arq / run_pair / options`.

Spectrum — `phy/python/channel_sense.py, freq_scan.py`

- `sense_channel(...)` — measure power at one frequency.
- `python3 phy/python/freq_scan.py --start 902 --stop 928 --rx-args addr=192.168.20.2` — find a quiet carrier.

8. Troubleshooting (beginner gotchas)

Symptom	Fix
pyphy import / Python.h / arch error (macOS) Exec format error ON sdr_system receiver gets nothing / no ACK startup "rate check" fails a stuck receiver won't stop	run via <code>./run.sh</code> (it sets <code>PYTHONPATH + arch -x86_64</code>), or build with <code>phy/bindings/build.sh</code> rebuild on that machine: <code>cd phy && cmake -S . -B build && cmake --build build</code> start the receiver first ; keep both radios on; use DQPSK or OFDM N210 needs <code>--rx-rate 2e6 --symbol-rate 1e6</code> ; set <code>--tx-rate = --rx-rate</code> press Ctrl-C again; last resort Ctrl-\ or kill -9

Symptom	Fix
coherent QPSK ~17% delivered	free-running clocks — use DQPSK (single-carrier) or OFDM

Beyond this guide

For deeper reference (in the repo root / applications/): **PARAMETERS.md** (every option — or run `sdr_system --help`), **SYSTEM_REFERENCE.pdf** (the engine math + every algorithm), **COMMANDS.md** and **USRP_CARRIER_MODULATION.txt** (ready-to-run command recipes per scheme / per device), **HARDWARE.md** (hardware setup), and **applications/EXPERIMENT_GUIDE.pdf** (running the example apps).